

**SWASTIK COLLEGE**

Tribhuvan University

Faculty of Humanities and Social Sciences



Bachelor of Computer Application

(BCA)

Course: Artificial Intelligence (CACS410)

Semester: 7<sup>th</sup>

A Lab Report On:

**Artificial Intelligence**

**Submitted by:**

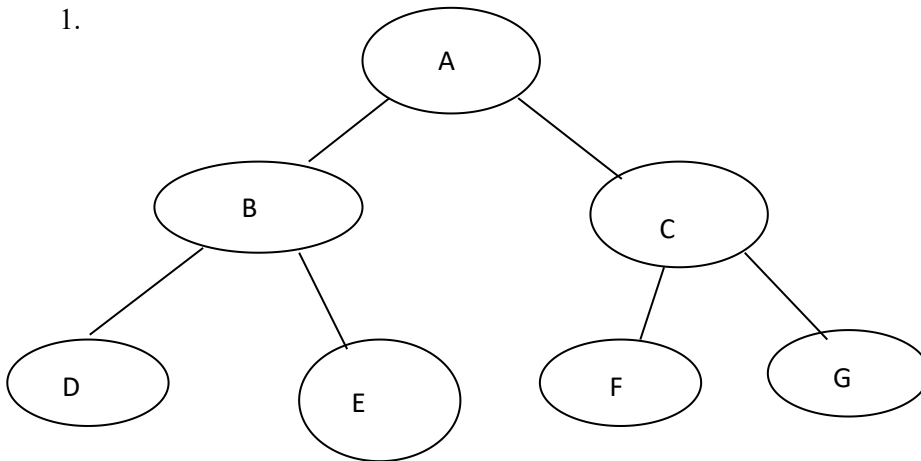
Name: Swosti Makaju

Roll No: 33

**Submitted to:**

Department of BCA

1.



- a. Draw above picture as output and implement BFS algorithm.
- b. Implement DFS algorithm.

- a. Draw above picture as output and implement BFS algorithm.

**Code:**

```
import networkx as nx

import matplotlib.pyplot as plt

from collections import deque

graph={'A':['B','C'],'B':['D','E'],'C':['F','G'],'D':[],'E':[],'F':[],'G':[]}

G=nx.DiGraph(graph)

pos={'A':(0,2),'B':(-1,1),'C':(1,1),'D':(-1.5,0),'E':(-0.5,0),'F':(0.5,0),'G':(1.5,0)}

nx.draw(G,pos,with_labels=True,node_color='lightblue',node_size=2000,arrows=True)

plt.show()

def bfs(graph,start):

    visited=[]; queue=deque([start])

    while queue:

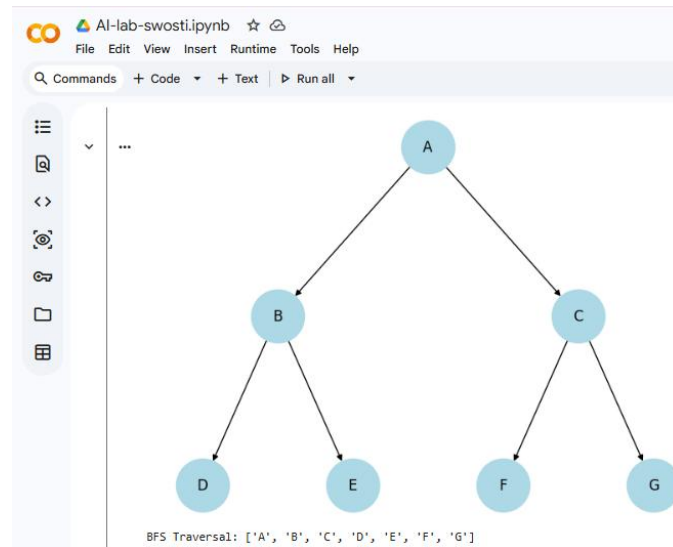
        node=queue.popleft()

        if node not in visited:    visited.append(node); queue.extend(graph[node])

    return visited

print("BFS Traversal:",bfs(graph,'A'))
```

## Output :



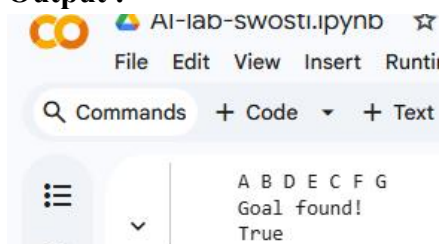
b. Implement DFS algorithm.

## Code:

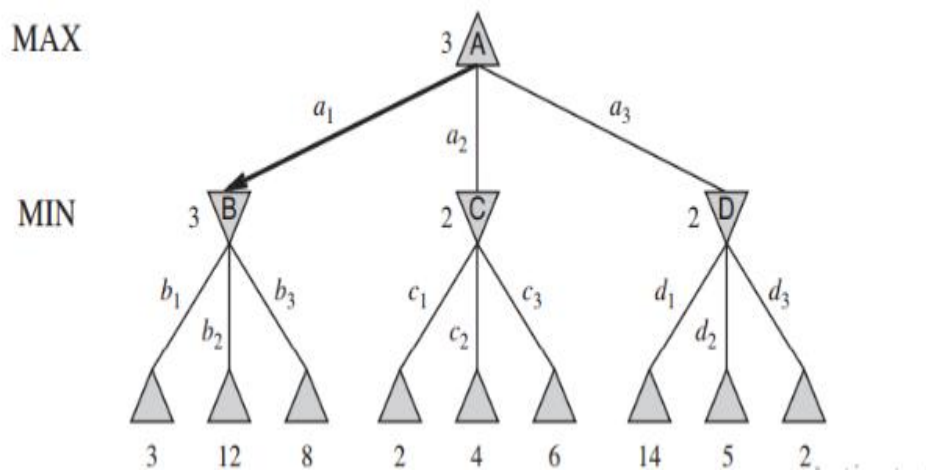
```
graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': [], 'E': [], 'F': [], 'G': [] }
```

```
def dfs(node, goal, visited=None):  
    if visited is None:    visited = []  
    print(node, end=" ")  
    visited.append(node)  
    if node == goal:    print("\nGoal found!")  
        return True  
    for neighbor in graph[node]:    if neighbor not in visited:  
        if dfs(neighbor, goal, visited):  
            return True  
    return False  
  
dfs('A', 'G')
```

## Output :



2.



Implement min max search tree algorithm on above figure.

**Code:**

```
tree = { 'A': { 'a1': { 'b1': 3, 'b2': 12, 'b3': 8 }, 'a2': { 'c1': 2, 'c2': 4, 'c3': 6 }, 'a3': { 'd1': 14, 'd2': 5, 'd3': 2 } } }
```

```
def minimax(node, is_maximizing):
```

```
    if isinstance(node, int):
```

```
        return node
```

```
    if is_maximizing:
```

```
        best_value = float('-inf')
```

```
        for action, child in node.items():
```

```
            value = minimax(child, False)
```

```
            best_value = max(best_value, value)
```

```
        return best_value
```

```
    else:
```

```
        best_value = float('inf')
```

```
        for action, child in node.items():
```

```
            value = minimax(child, True)
```

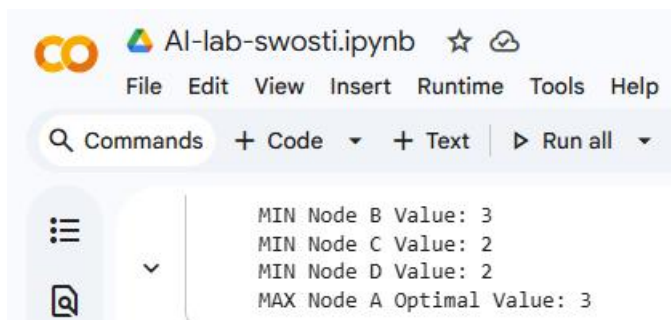
```
            best_value = min(best_value, value)
```

```
        return best_value

node_b = minimax(tree['A']['a1'], is_maximizing=False)
node_c = minimax(tree['A']['a2'], is_maximizing=False)
node_d = minimax(tree['A']['a3'], is_maximizing=False)
root_val = minimax(tree['A'], is_maximizing=True)

print(f"MIN Node B Value: {node_b}")
print(f"MIN Node C Value: {node_c}")
print(f"MIN Node D Value: {node_d}")
print(f"MAX Node A Optimal Value: {root_val}")
```

Output:



The screenshot shows a Jupyter Notebook interface with the file name 'AI-lab-swosti.ipynb'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a toolbar with a search icon, 'Commands', '+ Code', '+ Text', and a 'Run all' button. On the left side, there is a sidebar with a list icon and a search icon. The main area displays the output of the code execution:

```
MIN Node B Value: 3
MIN Node C Value: 2
MIN Node D Value: 2
MAX Node A Optimal Value: 3
```

3. Write program for implementing Naive Bayes.

**Code:**

```
import math

X = [[6.0, 180], [5.9, 190], [5.5, 150], [5.8, 170],
      [5.1, 110], [5.3, 120], [5.0, 100], [5.2, 115]]

y = ['Male', 'Male', 'Male', 'Male', 'Female', 'Female', 'Female', 'Female']

def calculate_likelihood(x, mean, var):

    eps = 1e-9

    exponent = math.exp(-((x - mean) ** 2) / (2 * (var + eps)))

    return (1 / math.sqrt(2 * math.pi * (var + eps))) * exponent

classes = list(set(y))

model = {}

for c in classes:

    X_c = [X[i] for i in range(len(y)) if y[i] == c]

    prior = len(X_c) / len(y)

    means = [sum(col) / len(col) for col in zip(*X_c)]

    variances = [sum((x - m) ** 2 for x in col) / len(col) for col, m in zip(zip(*X_c), means)]

    model[c] = {'prior': prior, 'means': means, 'variances': variances}

def predict(sample):

    posteriors = {}

    for c in classes:

        total_log_prob = math.log(model[c]['prior'])

        for i in range(len(sample)):

            mean = model[c]['means'][i]

            var = model[c]['variances'][i]

            prob = calculate_likelihood(sample[i], mean, var)

            total_log_prob += math.log(prob)
```

```
posterior[c] = total_log_prob

return max(posterior, key=posterior.get)

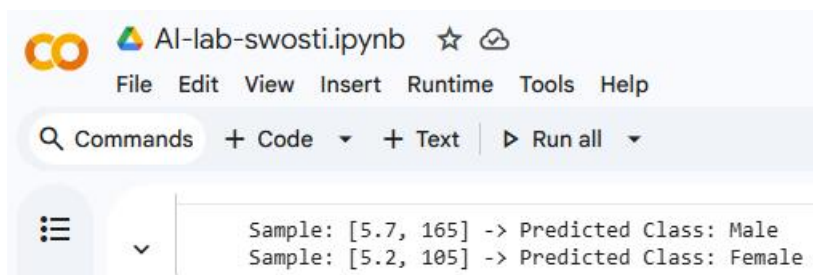
X_test = [[5.7, 165], [5.2, 105]]

for sample in X_test:

    prediction = predict(sample)

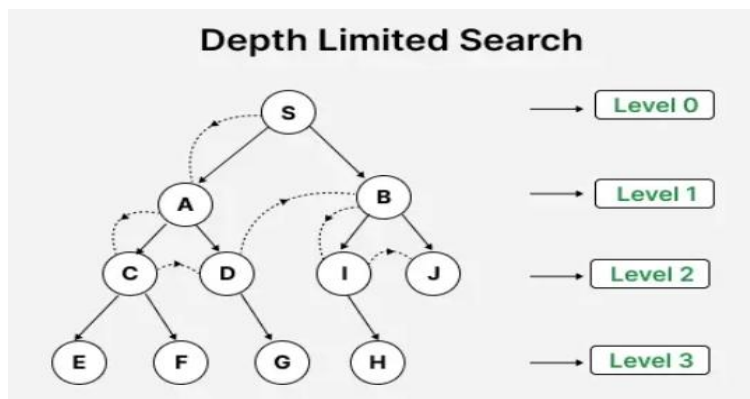
    print(f"Sample: {sample} -> Predicted Class: {prediction}")
```

### Output:

A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'AI-lab-swosti.ipynb' and icons for saving and starring. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar labeled 'Commands' is on the left, and buttons for '+ Code', '+ Text', and 'Run all' are on the right. The main area shows the output of the code: 'Sample: [5.7, 165] -> Predicted Class: Male' and 'Sample: [5.2, 105] -> Predicted Class: Female'.

```
AI-lab-swosti.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help
🔍 Commands + Code + Text ▶ Run all
⋮
Sample: [5.7, 165] -> Predicted Class: Male
Sample: [5.2, 105] -> Predicted Class: Female
```

4.



- a. Implement depth limit search upto level 2.
- b. Implement iterative deepening search upto level 3.

- a. Implement depth limit search upto level 2.

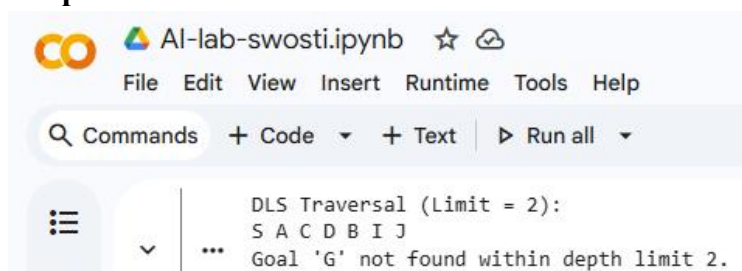
**Code:**

```
graph = {'S': ['A', 'B'], 'A': ['C', 'D'], 'B': ['I', 'J'], 'C': ['E', 'F'], 'D': ['G'], 'I': ['H'], 'E': [], 'F': [], 'G': [], 'H': [], 'J': [] }
```

```
def dls(node, goal, limit, depth=0):
    print(node, end=" ")
    if node == goal: return True
    if depth == limit: return False
    for neighbor in graph[node]:
        if dls(neighbor, goal, limit, depth + 1):
            return True
    return False

limit = 2
start_node = 'S'
goal_node = 'G'
print(f"DLS Traversal (Limit = {limit}):")
found = dls(start_node, goal_node, limit)
if found: print("\nGoal found!")
else:
    print(f"\nGoal '{goal_node}' not found within depth limit {limit}.")
```

**Output:**

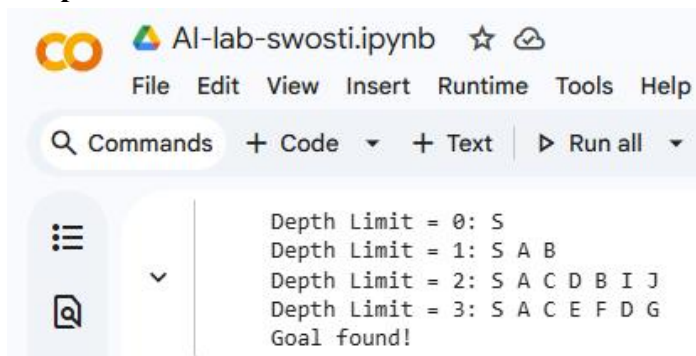


b. Implement iterative deepening search upto level3.

**Code:**

```
graph = {'S': ['A', 'B'], 'A': ['C', 'D'], 'B': ['T', 'J'], 'C': ['E', 'F'], 'D': ['G'], 'T': ['H'], 'E': [], 'F': [], 'G': [], 'H': [], 'J': [] }
def dls(node, goal, limit, depth=0):
    print(node, end=" ")
    if node == goal:
        return True
    if depth == limit:
        return False
    for neighbor in graph[node]:
        if dls(neighbor, goal, limit, depth + 1):
            return True
    return False
def ids(start, goal, max_limit):
    for limit in range(max_limit + 1):
        print(f"Depth Limit = {limit}: ", end="")
        if dls(start, goal, limit):
            print("\nGoal found!")
            return
    print()
    print("\nGoal not found within the maximum limit.")
ids('S', 'G', max_limit=3)
```

**Output:**

The screenshot shows a Jupyter Notebook window titled "AI-lab-swosti.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a search bar labeled "Commands" and buttons for "+ Code", "+ Text", and "Run all". On the left side, there is a sidebar with a list icon and a notebook icon. The main area displays the output of the code execution, which is a series of lines showing the search process at different depth limits. The output is: "Depth Limit = 0: S", "Depth Limit = 1: S A B", "Depth Limit = 2: S A C D B I J", "Depth Limit = 3: S A C E F D G", and "Goal found!".

```
Depth Limit = 0: S
Depth Limit = 1: S A B
Depth Limit = 2: S A C D B I J
Depth Limit = 3: S A C E F D G
Goal found!
```